

Group Name: Asteroid Destroyers

Team Members: Novali Plascencia, Emilia Mahmoodi, Andy Moughalian, Roee Palmon, Christian Rusanovsky

- Team tested : Team Rocket

- APIs tested :

<https://www.optcgapi.com/api/allSets/>

https://www.optcgapi.com/api/sets/card/{card_set_id}/

API tested: GET <https://www.optcgapi.com/api/sets/card/OP01-025/>

API clarity: The endpoint is self-documenting — [/sets/card/{card_set_id}/](#) clearly communicates intent. Field names follow consistent snake_case. The distinction between [set_id](#) (OP-01) and [card_set_id](#) (OP01-025) is logical once understood but could trip up new users.

Request/Response: Request is a simple GET with no auth, no body, no query params. Response returned 200 OK in 181ms at 1.54 KB — fast and lightweight. Response is a JSON array containing a single object, even though the query targets a unique card.

Error cases: Not tested in this run. Should verify behavior for invalid card IDs, malformed IDs, missing trailing slash, and non-existent sets.

Logic alignment: Data model is internally consistent. [life: null](#) correctly reflects that Characters don't have life values (only Leaders do). [counter_amount: 0](#) and pricing fields populated correctly.

Step 4: Written Testing Output

Field	Entry
Team tested	Self-test (no team partner assigned in this run)
APIs tested	GET /api/sets/card/OP01-025/

What is good	Fast response time (181ms), clean JSON structure, intuitive URL pattern, comprehensive card metadata including live pricing, no auth friction for read operations
Issues found	(1) Returns array instead of single object for unique-ID lookup. (2) No timestamp on price data — can't tell how stale <code>market_price</code> is. (3) Trailing slash appears required; unclear if non-trailing fails gracefully. (4) No documented rate limits visible in response headers
Suggestions	Return single object for unique queries; add <code>price_updated_at</code> timestamp; document error response shape; add <code>Cache-Control</code> headers; consider versioning the API path (<code>/v1/</code>)

Step 5: Individual Accountability

Explain your API: This API serves One Piece Trading Card Game data. The tested endpoint retrieves a single card's full metadata — gameplay stats, set info, and current market pricing — by its unique card-set identifier.

Show request/response:

Request:

GET <https://www.optcgapi.com/api/sets/card/OP01-025/>

Response (abbreviated):

json

```
[
  {
    "card_name": "Roronoa Zoro (025)",
    "card_set_id": "OP01-025",
    "rarity": "SR",
    "card_cost": "3",
    "card_power": "5000",
    "card_text": "[Rush] (This card can attack on the turn in which it is played.)",
    "inventory_price": 3.49,
    "market_price": 3.04
  }
]
```

Explain error case: If a client sends an invalid `card_set_id` (e.g., `0P01-999`), the expected behavior is a 404 with a structured error body. If the API instead returns 200 with an empty array, that's a logic flaw — clients can't distinguish "no results" from "endpoint not found" without extra checks.

What issue did your team find? The most consequential issue: pricing data has no freshness indicator. For a card pricing API where values fluctuate daily, omitting `price_updated_at` undermines the data's usefulness for any inventory or resale tooling built on top of it.

Step 6: Quick Feedback

- **Improve clarity:** Document whether trailing slashes are mandatory and what response shape errors take.
 - **Add error handling:** Standardize error responses — status code + `{ "error": "...", "code": "..." }` body — instead of relying on empty arrays.
 - **Align with project:** If this API is consumed by a card collection or trading app, the missing price timestamp is a blocker that should be raised with the API owner.
-

Step 7: Final Submission Checklist

- **Clean API:** Endpoint follows REST conventions, returns valid JSON, 200 OK
 - **JSON request/response:** Documented above with real captured values
 - **Testing results:** Captured in Step 4 table
 - **Individual contribution:** Solo test run; clarity, request/response, error reasoning, and team-level critique completed in Step 5
 - **Updated GitHub:** Not applicable to this run — would require committing the test report and Postman collection export to the repo
-

Step 8: Evaluation Notes

This submission demonstrates understanding of the API's structure, identifies non-obvious quality issues (price freshness, response-shape inconsistency), and

proposes concrete improvements. Participation evidence: full walkthrough of all required steps. Contribution: original analysis, not just data restatement.

Github links:

<https://github.com/emiliamah/summArIze>

<https://github.com/AndyMoughalian/COMP-491-L>

<https://github.com/ChristianCSUN/COMP-491-Final-Project>

<https://github.com/NovaPlas/summArIze>

<https://github.com/Astroid-Destroyers/summArIze/tree/alpha>